



ANÁLISIS TÉCNICO

Sistema de Clima Replicado de Pokémon GO

Especificación de implementación para aplicación meteorológica

Versión	1.0
Propósito	Especificación técnica de integración
Clasificación	Uso Interno

1. Resumen Ejecutivo

Este documento describe con precisión técnica cómo replicar el comportamiento del sistema de clima de Pokémon GO en una aplicación existente. El objetivo es que cuando el clima cambie en Pokémon GO, la aplicación refleje el mismo estado de forma sincronizada.

Los tres pilares del sistema son:

- Fuente de datos: Pronóstico horario de AccuWeather (no clima en tiempo real)
- Granularidad geográfica: Celdas S2 de Google (nivel 10)
- Cadencia de actualización: Consulta una vez por hora, en punto

⚠ IMPORTANTE: El mayor error a evitar es consultar el clima "actual" (current conditions). Pokémon GO usa el pronóstico para la hora vigente, lo que provoca diferencias de hasta 30-45 minutos con la realidad.

2. Fuente de Datos: AccuWeather

2.1 Endpoint requerido

Pokémon GO consume el pronóstico horario de AccuWeather. El endpoint relevante es:

```
GET https://dataservice.accuweather.com/forecasts/v1/hourly/12hour/{locationKey}
?apikey={API_KEY}&details=true&metric=true
```

Este endpoint devuelve los próximos 12 tramos horarios. Pokémon GO toma el tramo correspondiente a la hora actual (por ejemplo, si son las 15:32, utiliza el bloque 15:00–16:00).

2.2 Campos críticos del response

Campo JSON	Tipo	Uso en Pokémon GO
WeatherIcon	integer (1–44)	Mapeo directo al estado de clima del juego
IconPhrase	string	Solo referencia / debugging
Wind.Speed.Value	float (km/h)	Determina si se activa estado "Windy"
WindGust.Speed.Value	float (km/h)	Refuerzo para activar "Windy" por ráfagas
HasPrecipitation	boolean	Validación secundaria para lluvia/nieve
DateTime	ISO 8601	Identificación del bloque horario

2.3 Cadencia de consulta

La consulta a AccuWeather debe realizarse una sola vez por ciclo horario, no de forma continua. El momento óptimo es:

- Ejecutar la consulta a las HH:00 (en punto de cada hora)
- Como margen de seguridad, aceptar ejecuciones entre HH:00 y HH:10
- Cachear el resultado hasta el siguiente ciclo horario
- Si el usuario cambia de celda S2, invalidar el caché y consultar de nuevo

 **NOTA:** Consultar AccuWeather más de una vez por hora NO mejorará la sincronización con Pokémon GO, dado que el juego captura el pronóstico al inicio de cada hora y lo mantiene fijo durante todo ese periodo.

3. Geolocalización: Celdas S2

3.1 Concepto

Google S2 Geometry divide la superficie de la Tierra en celdas jerárquicas. Pokémon GO utiliza el nivel 10 de esta jerarquía para asignar un estado de clima único a cada área geográfica.

Características de las celdas S2 nivel 10:

- Tamaño aproximado: 200 km² (varía según latitud)
- Forma: cuadrilátera, con lados de ~14 km

- Cada celda tiene un centro geográfico único (lat/lng) que es el punto de consulta a AccuWeather

3.2 Librería recomendada por plataforma

Plataforma	Librería	Instalación
JavaScript / Node.js	s2-geometry	npm install s2-geometry
Python	s2sphere	pip install s2sphere
iOS (Swift)	s2-swift	Swift Package Manager
Android (Kotlin)	s2-geometry-library-java	Via Maven/Gradle
Web (sin librería)	Algoritmo manual L10	Ver sección 3.3

3.3 Implementación en JavaScript

Obtener el centro de la celda S2 nivel 10 a partir de las coordenadas del usuario:

```
import { S2 } from 's2-geometry';

function getS2CellCenter(lat, lng) {
  const key = S2.latLngToKey(lat, lng, 10); // nivel 10
  const cellId = S2.keyToId(key);
  const latLng = S2.idToLatLng(cellId);
  return { lat: latLng.lat, lng: latLng.lng };
}

// Uso: consultar AccuWeather con el centro de la celda, NO con la posición real
const center = getS2CellCenter(userLat, userLng);
fetchAccuWeatherForecast(center.lat, center.lng);
```

3.4 Caching por celda S2

Cada celda S2 tiene su propio estado de clima independiente. El sistema de caché debe indexarse por clave de celda, no por coordenadas exactas:

```
// Estructura de caché recomendada
const weatherCache = {
  '4b59a394': { // clave S2 nivel 10
    state: 'RAINY',
    fetchedAt: 1704067200, // timestamp Unix en punto de hora
    expiresAt: 1704070800 // fetchedAt + 3600 segundos
  }
}
```

4. Mapeo AccuWeather → Estados de Clima

4.1 Los 7 estados de Pokémon GO

Pokémon GO define exactamente 7 condiciones meteorológicas. Cada una se mapea desde el campo WeatherIcon de AccuWeather:

Estado GO	Emoji	Icon IDs AccuWeather	Descripción
SUNNY	☀️ / 🌤️	1, 2, 3, 4, 30, 33, 34	Soleado, mayormente soleado, caluroso, despejado nocturno
PARTLY_CLOUDY	🌤️	5, 6, 35, 36	Parcialmente nublado, mayormente despejado
CLOUDY	☁️	7, 8, 11, 37, 38	Nublado, muy nublado, niebla ligera
RAINY	🌧️	12, 13, 14, 15, 16, 17, 40, 41, 42	Lluvia, chubascos, tormentas eléctricas
SNOW	❄️	19, 20, 21, 22, 23, 24, 25, 26, 29, 43, 44	Nieve, cellisca, aguanieve, hielo
FOG	🌫️	11 (con visibilidad < 1km)	Niebla densa
WINDY	🌪️	SUPERPUESTO (ver 4.2)	Se activa por velocidad de viento

4.2 Lógica especial: Estado WINDY

"Windy" no es un estado exclusivo basado en un icon ID. Es una capa que se superpone a otros estados según las condiciones de viento:

```
function resolveWeatherState(iconId, windSpeedKmh, windGustKmh) {
  const baseState = getBaseState(iconId); // SUNNY, CLOUDY, etc.

  const isWindy = windSpeedKmh >= 24.1 || windGustKmh >= 35.0;

  // WINDY reemplaza el estado base solo si la base es SUNNY o PARTLY_CLOUDY
  if (isWindy && ['SUNNY', 'PARTLY_CLOUDY', 'CLOUDY'].includes(baseState)) {
    return 'WINDY';
  }

  // RAINY/SNOW/FOG tienen prioridad sobre WINDY
  return baseState;
}
```

 **IMPORTANTE:** Los umbrales exactos son: velocidad sostenida ≥ 24.1 km/h (15 mph) O ráfagas ≥ 35.4 km/h (22 mph). Si solo se dispone de uno de los dos valores, aplicar el que esté disponible.

4.3 Función de mapeo completa (JavaScript)

```
const ACCUWEATHER_TO_POKEMON = {
  SUNNY:      [1, 2, 3, 4, 30, 33, 34],
  PARTLY_CLOUDY: [5, 6, 35, 36],
  CLOUDY:     [7, 8, 38],
  FOG:        [11, 37],
  RAINY:      [12, 13, 14, 15, 16, 17, 18, 40, 41, 42],
  SNOW:       [19, 20, 21, 22, 23, 24, 25, 26, 29, 43, 44],
};

function getBaseState(iconId) {
  for (const [state, ids] of Object.entries(ACCUWEATHER_TO_POKEMON)) {
    if (ids.includes(iconId)) return state;
  }
  return 'PARTLY_CLOUDY'; // fallback
}
```

5. Clima Extremo

Pokémon GO define un estado adicional de "Extreme Weather" que desactiva bonificaciones. Se activa por alertas meteorológicas de AccuWeather, no por condiciones de temperatura o viento per se.

5.1 Detección

```
// Endpoint de alertas de AccuWeather
GET /alerts/v1/{locationKey}?apikey={KEY}&details=true

// Condición: cualquier alerta activa activa el modo extremo
function isExtremeWeather(alerts) {
  return alerts && alerts.length > 0;
}
```

- Las alertas incluyen: calor extremo, frío extremo, viento fuerte, inundaciones, tormentas severas
- Una alerta leve (p.ej. "oleaje inusual") también puede activar este estado
- La consulta de alertas puede hacerse en el mismo ciclo horario que el pronóstico

6. Flujo de Integración Completo

6.1 Diagrama de flujo

El siguiente pseudocódigo describe el flujo completo que debe ejecutarse en cada ciclo horario:

```
async function updateWeather(userLat, userLng) {
```

```
// 1. Obtener celda S2 nivel 10
const s2Key = getS2Key(userLat, userLng, level=10);

// 2. Verificar caché
const cached = weatherCache.get(s2Key);
const currentHour = Math.floor(Date.now() / 3600000);
if (cached && cached.hour === currentHour) return cached.state;

// 3. Obtener centro de celda S2
const center = getS2CellCenter(s2Key);

// 4. Obtener locationKey de AccuWeather por coordenadas
const locKey = await getAccuWeatherLocationKey(center.lat, center.lng);

// 5. Consultar pronóstico horario
const forecast = await getHourlyForecast(locKey);
const current = forecast[0]; // primer bloque = hora vigente

// 6. Resolver estado
const base = getBaseState(current.WeatherIcon);
const wind = current.Wind.Speed.Value;
const gust = current.WindGust.Speed.Value;
const state = resolveWeatherState(current.WeatherIcon, wind, gust);

// 7. Verificar clima extremo
const alerts = await getAlerts(locKey);
const isExtreme = isExtremeWeather(alerts);

// 8. Guardar en caché
weatherCache.set(s2Key, { state, isExtreme, hour: currentHour });

return { state, isExtreme };
}
```

6.2 Gestión de la Location Key de AccuWeather

AccuWeather no acepta coordenadas directamente en el endpoint de pronóstico. Se requiere un Location Key previo:

```
// Paso previo: obtener Location Key por coordenadas
GET /locations/v1/cities/geoposition/search
?apikey={KEY}&q={lat},{lng}&tolevel=true
```

La Location Key para el centro de una celda S2 es estable — no cambia. Se puede cachear indefinidamente en una base de datos local para evitar llamadas extra a la API.

 **RECOMENDACIÓN:** Cachear las Location Keys por clave S2 en base de datos local ahorra una llamada extra de API por cada usuario en cada ciclo horario. Esta optimización es relevante a escala.

7. Plan de Implementación

#	Tarea	Prioridad	Consideraciones
1	Integrar librería S2 en el proyecto	Alta	Elegir librería según plataforma (ver sec. 3.2)
2	Crear función getS2CellCenter(lat, lng)	Alta	Devuelve lat/lng del centro de celda nivel 10
3	Sustituir endpoint de clima actual por hourly forecast	Alta	Cambio en la URL de AccuWeather
4	Implementar tabla de mapeo WeatherIcon → estado	Alta	Ver sección 4.1 y 4.3
5	Añadir lógica de detección WINDY por viento	Alta	Umbrales: ≥ 24.1 km/h o ráfagas ≥ 35.4 km/h
6	Implementar caché horario indexado por clave S2	Media	Evita reconsultas innecesarias
7	Cachear Location Keys AccuWeather en BD local	Media	Reducción de llamadas a API
8	Consultar endpoint de alertas para clima extremo	Media	Ver sección 5
9	Implementar invalidación de caché al cambiar de celda S2	Media	Detectar cruce de frontera S2
10	QA: comparar estado con Pokémon GO en tiempo real	Baja	Validar sincronización en distintas ciudades

8. Consideraciones y Limitaciones

8.1 Por qué puede haber desincronización residual

- AccuWeather puede actualizar su pronóstico durante la hora vigente. Pokémon GO ya capturó el pronóstico al inicio; la app lo hará también al inicio del ciclo, por lo que ambos deberían coincidir.
- En zonas rurales o con cobertura climática limitada, AccuWeather puede aproximar condiciones desde estaciones lejanas, creando diferencias locales.
- Si la aplicación usa otra clave de API de AccuWeather con diferente nivel de servicio, los icon IDs pueden variar ligeramente entre regiones.

8.2 Diferencias irreducibles

- Las celdas S2 no respetan accidentes geográficos (montañas, ríos, costas). Dos puntos muy cercanos en celdas distintas pueden tener climas distintos en el juego.
- Pokémon GO aplica suavizado interno entre estados para evitar cambios bruscos. Replicar este suavizado es muy complejo y su impacto visual es menor.

☑ **RECOMENDACIÓN:** Con esta implementación se puede alcanzar una sincronización del 95%+ con el clima mostrado en Pokémon GO en la misma ubicación, con las excepciones descritas en la sección 8.1 como margen residual.